

Alpamayo-R1 VRAM 최적화 — 스왑 방식 아이디어 정리

날짜

2026-02-26

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

개요

모델 구조 (BF16 기준)

방법 1: 레이어 단위 스왑 (On-Demand Layering)

개념

구현 상태: [완료]

실험 결과

한계

방법 2: 최적 청크 단위 전송

개념

구현 상태: [벤치마크 완료]

실험 결과

활용 방안

방법 3: 모듈 단위 스왑 → [실험적 불가 판정]

원래 개념

검증 결과: 불가능

실제 구현된 방식 (하이브리드)

방법 4: 비동기 2-Stream 파이프라인 (Prefetch + Free)

핵심 통찰: D2H는 불필요

개념

구현 방식

기대 효과

선행 연구 비교

연구 포지셔닝

참고 문헌

Alpamayo-R1 VRAM 최적화 — 스왑 방식 아이디어 정리

2026-02-26

개요

Alpamayo-R1-10B 모델(BF16 ~20.65GB)을 12GB VRAM GPU에서 추론하기 위한 스왑 방식을 정리한다. 기존 실험에서 검증된 3가지 방법과 추가 제안된 1가지 방법을 비교·분석한다.

모델 구조 (BF16 기준)

모듈	레이어 수	크기
Vision Encoder	—	1.15 GB
VLM (Qwen3-VL-8B)	36	15.17 GB
Expert	36	4.56 GB
합계	—	~20.65 GB

방법 1: 레이어 단위 스왑 (On-Demand Layering)

개념

개별 Transformer 레이어를 forward hook으로 GPU↔CPU 간 이동시킨다. 레이어 실행 직전에 GPU로 로드하고, 실행 완료 후 CPU로 언로드한다.

구현 상태: [완료]

- 구현: `research/06-demand-layering-impl/demand_layering.py`
- VLM 36개 레이어에 `register_forward_pre_hook()` / `register_forward_hook()` 적용

- 레이어당 크기: ~0.386 GB (192.9M params)

실험 결과

항목	값
추론 시간	116.51s
피크 VRAM	11.03 GB
H2D 전송	450회, 평균 64.23 ms/layer
D2H 전송	450회, 평균 156.72 ms/layer
vs FP16 Unified Memory	2.35x 빠름 (273.79s → 116.51s)

한계

- 동기식 전송: 레이어 로드 대기 시간이 추론에 직접 반영
- 불필요한 D2H 수행: 현재 구현은 `module.to("cpu")` 로 D2H copy를 수행하나, 파라미터 원본은 CPU에 이미 존재하므로 GPU 메모리 해제(free)만 하면 됨. D2H 70.5s는 완전히 제거 가능한 오버헤드
- PCIe 유휴 시간 多 — 계산과 전송이 겹치지 않음

방법 2: 최적 청크 단위 전송

개념

레이어를 2-8MB 청크로 분할하여 전송함으로써 PCIe 대역폭을 극대화한다. 단일 대용량 전송보다 소규모 청크가 더 높은 유효 대역폭을 달성한다.

구현 상태: [벤치마크 완료]

- 실험: `research/03-swap-optimization/`
- Pinned memory 전송 벤치마크 수행 (1MB ~ 1024MB)

실험 결과

체크 크기	유효 대역폭	비고
2-8 MB	12.16-12.41 GB/s	최적 구간
16 MB	12.08 GB/s	소폭 감소
512 MB (단일)	7.77 GB/s	기준

- 최적 체크: 단일 전송 대비 **1.6x 대역폭 향상**
- PCIe Gen3 x16 이론 최대: 15.75 GB/s → 실측 12.4 GB/s (79% 활용)
- WSL2 전송당 고정 오버헤드: 0.36ms
- 대역폭 비대칭: Pinned H2D 8.5 GB/s / Pinned D2H 11.8 GB/s

활용 방안

방법 1의 레이어 전송을 체크 단위로 세분화하여 PCIe 효율을 높일 수 있다.

방법 3: 모듈 단위 스왑 → [실험적 불가 판정]

원래 개념

VLM↔Expert를 통째로 스왑하고, VLM 내부는 하이브리드로 처리한다.

검증 결과: 불가능

- 실험: `research/02-memory-pipelining/test_sequential_offload.py`
- VLM 단독 15.17GB > 12GB VRAM — 모듈 단위 로드 자체가 불가

Vision Encoder:	1.15 GB	→ 0	GPU 적재 가능
VLM:	15.17 GB	→ X	12GB 초과, 모듈 단위 불가
Expert:	4.56 GB	→ 0	GPU 적재 가능

실제 구현된 방식 (하이브리드)

모듈	스왑 방식	이유
Vision Encoder	모듈 단위	1.15GB — GPU에 전체 적재 가능
VLM	레이어 단위	15.17GB — 모듈 단위 불가
Expert	GPU 상주	4.56GB — KV Cache 접근 필요

결론: 순수 모듈 단위 스왑은 불가능하며, 방법 1(레이어 단위)이 필수적이다.

방법 4: 비동기 2-Stream 파이프라인 (Prefetch + Free)

핵심 통찰: D2H는 불필요

파라미터 오프로딩 추론에서 GPU→CPU 복사(D2H)는 필요 없다. 파라미터 원본은 CPU 메모리에 이미 존재하므로, 사용 완료된 레이어는 **GPU 메모리를 해제(free)하기만 하면 된다.**

D2H가 필요한 경우: Activation 저장 (학습), KV Cache 보존
D2H가 불필요한 경우: 파라미터 오프로딩 (추론) ← 우리 시나리오

현재 구현(`demand_layering.py`)의 `module.to("cpu")` 는 불필요한 D2H copy를 수행하고 있으며, 이는 추론 시간의 60% (70.5s/116.51s)를 차지하는 낭비이다.

개념

2개의 CUDA 스트림으로 계산과 H2D prefetch를 겹친다.

시간 →
 Compute Stream: [Layer N 계산] [Layer N+1 계산] [Layer N+2 계산]
 H2D Stream: [Layer N+1 로드] [Layer N+2 로드] [Layer N+3 로드]
 + Layer N-1 free + Layer N free + Layer N+1 free

- **Compute Stream:** 현재 레이어 forward pass 실행
- **H2D Stream:** 다음 레이어를 GPU로 prefetch (pinned memory → GPU)
- **GPU free:** 사용 완료된 레이어 메모리 즉시 해제 (copy 없음, 비용 ~0)

구현 방식

```
# CPU에 파라미터 원본을 pinned memory로 유지 (read-only)
cpu_params = {name: param.pin_memory() for name, param in layer.named_parameters()}

# GPU 버퍼에 H2D copy (비동기)
with torch.cuda.stream(h2d_stream):
    gpu_buffer = {k: v.to("cuda", non_blocking=True) for k, v in cpu_params.items()}

# 계산 완료 후: free만 수행 (D2H 불필요)
del gpu_buffer # GPU 메모리 즉시 해제, CPU 원본은 그대로 유지
```

기대 효과

방법 1 대비 두 가지 개선:

1. **D2H 제거**: 70.5s $\rightarrow$ 0s (추론 시간의 60% 절감)
2. **비동기 prefetch**: H2D 28.9s를 계산과 겹쳐서 은닉

이론적 최선: 추론 시간 = max(총 계산 시간, 총 H2D 시간)

시나리오	추론 시간 (추정)	vs 현재
현재 (동기 H2D + 불필요 D2H)	116.51s	baseline
D2H 제거만 (동기 H2D)	~46s	2.5x
D2H 제거 + 비동기 prefetch	~29-35s	3.3-4x

선행 연구 비교

기존 연구들이 D2H를 사용하는 이유는 **학습(backward pass)** 또는 **KV Cache 보존**이 필요한 시나리오이기 때문이다. 순수 파라미터 오프로딩 추론에서는 H2D + free만으로 충분하다.

시스템	연도	H2D Prefetch	D2H 사용 이유	대상
SwapAdvisor (ASPLOS)	2020	O	Activation 저장 (학습)	학습
Demand Layering (RTSS)	2022	O	SSD writeback	RT 추론
FlexGen (ICML)	2023	O	KV Cache + Activation offload	배치 추론
LightX2V	2025	O	중간 상태 보존	비디오 생성
PIPO	2025	O	KV Cache saving	소비자 추론
본 연구	2026	O	불필요 (free만)	VLA 추론

연구 포지셔닝

- 자율주행 VLA 모델에 대한 **도메인 특화 적용 및 실증 연구**
- **D2H 제거 + 2-Stream 단순 설계**의 실효성 검증
- 소비자 GPU(12GB)에서의 **pinned memory 기반 최적 prefetch 분석**
- 이기종 모듈 크기(1.15 / 15.17 / 4.56 GB)에 대한 **실용적 구현 및 벤치마크**

참고 문헌

- SwapAdvisor — Huang et al., ASPLOS 2020
- Demand Layering — Bae et al., RTSS 2022 (arXiv:2210.04024)
- Harmony — Li et al., VLDB 2022
- FlexGen — Sheng et al., ICML 2023 (arXiv:2303.06865)
- vDNN — Rhu et al., 2016 (arXiv:1602.08124)
- Capuchin — Peng et al., ASPLOS 2020
- LightX2V — 2025
- PIPO — 2025 (arXiv:2504.03664)